AGENTS

> Base agent rules: `constitution/AGENTS.md` — READ IT FIRST. The base file is authoritative for any topic not covered here. Project-specific rules below extend them; they never weaken them.
>
> Locate the constitution submodule from any nested depth with its `constitution/find_constitution.sh` helper. Canonical reference: https://github.com/HelixDevelopment/HelixConstitution

This file contains project-specific information intended for AI coding agents. The reader is expected to know nothing about the project beforehand.

Project Overview

This is a Go-based ebook translation system that translates books between multiple languages using various LLM providers. It supports multiple input/output formats (FB2, EPUB, TXT, HTML, PDF, DOCX) and provides both CLI tools and API servers for translation workflows.

Key capabilities:

- Multi-format ebook parsing and generation
- Integration with multiple LLM providers (OpenAI, Anthropic, DeepSeek, Zhipu, Qwen, Gemini, Ollama, LlamaCpp)
- Distributed translation via SSH workers
- Real-time WebSocket monitoring dashboard
- REST API and gRPC interfaces
- Translation caching, quality verification, and multi-pass processing
- Preparation phase analysis for improved translation quality
- Markdown workflow (EPUB to Markdown and back)

Technology Stack

- **Language**: Go 1.25.2 (per `go.mod`); `Dockerfile` uses `golang:1.24-alpine` as builder
- **Module**: `digital.vasic.translator`
- **Version**: Makefile declares `3.0.0`; `VERSION` file reads `2.3.0`
- **Web Framework**: Gin (github.com/gin-gonic/gin)
- **WebSocket**: Gorilla WebSocket (github.com/gorilla/websocket)
- **gRPC**: google.golang.org/grpc with Protocol Buffers

- **Authentication**: JWT (github.com/golang-jwt/jwt/v5)
- **Databases**: PostgreSQL (github.com/lib/pq), SQLite (github.com/mattn/go-sqlite3), Redis (github.com/redis/go-redis/v9)
- **Document Processing**: unidoc/unioffice, unidoc/unipdf/v3
- **CLI Framework**: Cobra (github.com/spf13/cobra)
- **Testing**: testify (github.com/stretchr/testify)
- **Transport**: QUIC support (github.com/quic-go/quic-go)
- **TLS**: golang.org/x/crypto

Build and Test Commands

Building

```
# Build all primary binaries (gRPC server, API server, unified
        CLI)
make build

# Build specific components
make build-grpc      # ./build/grpc-server
make build-api       # ./build/api-server
make build-cli       # ./build/unified-translator

# Build for all platforms (Linux AMD64/ARM64, macOS AMD64/ARM64,
        Windows AMD64)
make build-all

# Clean build artifacts
make clean
```

Additional entry points that can be built individually:

```
go build -o build/monitor-server ./cmd/monitor-server
go build -o build/translator ./cmd/translator
go build -o build/server ./cmd/server
go build -o build/markdown-translator ./cmd/markdown-translator
go build -o build/preparation-translator ./cmd/preparation-
        translator
go build -o build/ebook-translator ./cmd/ebook-translator
go build -o build/translate-ssh ./cmd/translate-ssh
go build -o build/deployment ./cmd/deployment
```

Testing

```
# Run all tests (excludes build-tagged tests by default)
make test
# or
go test ./... -v

# Run tests with coverage (generates coverage.html)
make test-coverage
```

```
# Run a specific package
go test -v ./pkg/translator
go test -v ./pkg/events
go test -v ./pkg/websocket

# Run a specific test function
go test -v -run TestFunctionName ./pkg/package

# Quick development cycle (format + vet + test)
make quick-test

# Pre-commit checks
make pre-commit
```

Running Tagged Tests

Tests outside the default cycle are gated by build tags:

```
# Integration tests
go test -tags=integration -v ./test/integration/...

# End-to-end tests
go test -tags=e2e -v ./test/e2e/...

# Performance benchmarks
go test -tags=performance -v ./test/performance/...

# Stress tests
go test -tags=stress -v ./test/stress/...

# Security tests
go test -tags=security -v ./test/security/...
```

Code Quality

```
# Format code
make fmt

# Vet code
make vet

# Lint (requires golangci-lint installation)
golangci-lint run
# Configuration is in .golangci.yml
```

Running Services (Development)

```
# Start development environment (gRPC on :50051 + API on :8080 in
        debug mode)
make dev
```

```
# Run full system (builds and runs both servers)
make run-system

# Individual servers
make run-grpc    # gRPC server on port 50051
make run-api     # API server on port 8080

# WebSocket monitoring server
go run ./cmd/monitor-server
# Dashboard available at http://localhost:8090/monitor
```

Docker

```
# Build Docker image
make docker-build

# Run Docker container
make docker-run

# Full stack with PostgreSQL and Redis
docker-compose up -d
```

Code Organization

Directory Structure

```
cmd/                    # Command-line entry points (14 binaries)
  api-server/          # REST API server (port 8080)
  challenge-runner/    # Runs project challenges/validations
  cli/                 # Legacy CLI tool
  deployment/          # Deployment management tool
  ebook-translator/    # Specialized SSH ebook translator
  grpc-server/         # gRPC translation service (port 50051)
  markdown-translator/ # Markdown workflow tool
  monitor-server/      # WebSocket monitoring server (port 8090)
  preparation-translator/ # Pre-translation analysis tool
  server/              # Main REST API server (older entry point,
TLS/HTTP3)
  ssh-translation/     # Complete SSH-based translation system
  translate-ssh/       # SSH worker standalone binary
  translator/          # Legacy translator CLI
  unified-translator/  # Primary unified CLI with full provider
support

pkg/                    # Public packages
  api/                 # REST API handlers, middleware, routing
  batch/               # Batch processing logic
  coordination/        # Multi-LLM coordination
  deployment/          # Docker and SSH deployment utilities
```

```
  distributed/         # Distributed processing over SSH workers
  ebook/               # Universal ebook parsing (FB2, EPUB, TXT,
HTML, PDF, DOCX)
  events/              # Event bus system for real-time updates
  fb2/                 # FB2 format-specific handling
  format/              # Format detection and validation
  grpc/                # gRPC service implementation and proto
definitions
  hardware/            # Hardware detection for optimization
  hash/                # Codebase hash utilities
  language/            # Language detection utilities
  logger/              # Structured logging utilities
  markdown/            # EPUB to Markdown conversion workflow
  models/              # Data models and registry
  preparation/         # Pre-translation preparation phase
  progress/            # Progress tracking
  report/              # Report generation
  script/              # Script conversion (Cyrillic/Latin)
  security/            # JWT authentication, rate limiting
  sshworker/           # SSH worker management
  storage/             # Database abstraction (PostgreSQL, Redis,
SQLite)
  translator/          # Translation engine core
    llm/               # LLM provider implementations
  verification/        # Translation quality verification
  version/             # Version information
  websocket/           # WebSocket hub and connection management

internal/               # Internal packages
  cache/               # In-memory caching layer
  config/              # Configuration loading and validation
  scripts/             # Production and deployment shell scripts
  working/             # Working configuration files and runtime
data

test/                   # Test suites organized by scope
  distributed/         # Distributed system tests
  e2e/                 # End-to-end tests
  fixtures/            # Test data (ebooks, configs, translations)
  integration/         # Cross-package integration tests
  mocks/               # testify/mock implementations
  performance/         # Benchmarks and performance tests
  security/            # Security-focused tests
  stress/              # Load and stress tests
  translator/          # Translator-specific tests
  unit/                # Unit tests for individual components
  utils/               # Test helper utilities and infrastructure

api/                    # API documentation
  examples/            # API usage examples
  openapi/             # OpenAPI 3.0 specification

web/                    # Web interface assets
  templates/           # HTML templates (dashboard.html)
```

```
scripts/                # Utility and demonstration scripts
  host-power-management/  # CONST-033 enforcement scripts
  demo-all.sh
  ebook_translation_workflow.sh
  python_translation.sh
  run_monitoring_demo.sh

build/                  # Build artifacts directory
docs/                   # Project documentation
Documentation/          # Additional documentation and guides
challenges/             # Submodule: challenge framework
containers/             # Submodule: container runtime extensions
```

Code Style Guidelines

Go Conventions

- **Naming**: PascalCase for exported identifiers, camelCase for unexported.
- **Imports order**: Standard library → third-party → local packages (`digital.vasic.translator/...`). Keep alphabetical within each group.
- **Types**: Both `interface{}` and `any` appear in the codebase; `interface{}` is still prevalent in many packages (e.g., `map[string]interface{}` for config options and event data).
- **Error handling**: Always handle errors explicitly. Wrap errors with context: `fmt.Errorf("operation failed: %w", err)`.
- **Comments**: Document all exported functions, types, and packages. Avoid commenting the obvious.
- **Line length**: Keep lines under 140 characters (per `.golangci.yml`).
- **Function length**: Aim for under 150 lines or 120 statements (`funlen` setting). Tests and `cmd/` entry points have relaxed rules.
- **Cyclomatic complexity**: `gocyclo` minimum is 35 in config; `gocognit` minimum is 20.

Security Rules

- **Never hardcode API keys or secrets**. Use environment variables or secure config files.
- **Never commit files containing secrets**. `.gitignore` already excludes `config_with_keys.json`, `.env`, `secrets.*`, etc.
- Input validation is required on all API endpoints.
- Use parameterized queries or ORM patterns for database access.

Linting Configuration

The `.golangci.yml` file enables a strict set of linters including:

- `govet`, `staticcheck`, `gosimple`, `ineffassign` for correctness
- `gocyclo`, `funlen`, `gocognit`, `nestif` for complexity
- `gosec` for security issues

- `goimports` with local prefix `digital.vasic.translator`
- `misspell` with US English locale

Test files and `cmd/` entry points have relaxed rules for `funlen`, `gochecknoinits`, `gosec`, `errcheck`, etc.

Testing Strategy

Test Organization

Tests are organized by scope rather than always co-located with source files:

- `test/unit/` - Unit tests for individual packages and components
- `test/integration/` - Cross-package integration tests (build tag: `integration`)
- `test/e2e/` - End-to-end tests with realistic scenarios (build tag: `e2e`)
- `test/security/` - Authentication, authorization, and security tests (build tag: `security`)
- `test/performance/` - Benchmarks and performance tests (build tag: `performance`)
- `test/stress/` - Load and stress tests (build tag: `stress`)
- `test/distributed/` - Distributed system and SSH worker tests
- `test/mocks/` - Shared mock implementations using testify/mock
- `test/fixtures/` - Sample ebooks, configs, and translation data
- `test/utils/` - Shared test helpers and infrastructure

Many packages also have `*_test.go` files alongside source code, particularly in `pkg/api/`, `pkg/translator/`, `pkg/distributed/`, `cmd/`, and `internal/`.

Testing Patterns

- **Table-driven tests**: Preferred style. Define a slice of anonymous structs with inputs and expected outputs, then iterate with `t.Run()`.
- **Sub-tests**: Use `t.Run("descriptive_name", func(t *testing.T) { ... })` for clarity.
- **Mocking**: Use `testify/mock` for interface mocking. Mocks are defined in `test/mocks/providers.go`.
- **Build tags**: Use `//go:build integration`, `//go:build e2e`, `//go:build performance`, `//go:build stress`, `//go:build security` for tests that should not run in the default quick-test cycle.
- **HTTP testing**: Use `httptest` package and `gin.SetMode(gin.TestMode)` for API handler tests.
- **SSH testing**: `test/utils/ssh_test_server.go` provides a full in-process SSH server with generated RSA keys for distributed tests.
- **Short mode**: Many tests check `testing.Short()` and skip when `-short` is used.

Test Utilities

- `test/utils/helpers.go` provides helpers for creating temporary test files, test configs, and minimal valid ebooks (EPUB, FB2, TXT, HTML).
- `test/utils/test_infrastructure.go` provides `TestHTTPServer` and `TestWebSocketServer` for integration tests.
- `test/utils/ports.go` manages dynamic port allocation starting from 30000.
- `test/mocks/providers.go` provides `MockTranslator`, `MockLLMProvider`, `MockDatabase`, `MockSecurityProvider`, `MockStorage`, and `MockProgressReporter`.

Coverage

- **Tool**: Go's built-in coverage (`go test -cover`)
- **Makefile target**: `make test-coverage`
- **Artifacts**: `coverage.out` and `coverage.html` are generated at project root
- **Current coverage**: Approximately 43.6% overall

Architecture and Key Patterns

Event-Driven Architecture

The system uses a central event bus (`pkg/events/events.go`) for component communication and real-time progress tracking:

- **Event types**: `EventTranslationStarted`, `EventTranslationProgress`, `EventTranslationCompleted`, `EventTranslationError`, `EventConversionStarted`, `EventConversionProgress`, `EventConversionCompleted`, `EventConversionError`.
- **Subscription**: `Subscribe(eventType, handler)` for type-specific; `SubscribeAll(handler)` for global listeners.
- **WebSocket Integration**: `pkg/websocket/hub.go` subscribes to all events and broadcasts to connected dashboard clients.
- **Session tracking**: Every translation operation has a unique `SessionID`. Events are tagged with session IDs so the dashboard can filter per-client.
- **Concurrency**: Every event handler is invoked in its own goroutine with panic recovery.

Translation Pipeline

1. **Format detection** (`pkg/format/detector.go`) identifies the input file type by magic bytes, extension, and content analysis.
2. **Parsing** (`pkg/ebook/parser.go` and format-specific parsers) extracts content into a `Book` model.
3. **Preparation** (optional, `pkg/preparation/`) analyzes content for characters, terminology, culture, and chapter structure using multi-pass LLM analysis.
4. **Translation** (`pkg/translator/translator.go` and `pkg/translator/llm/`) sends segments to LLM providers with smart retry and automatic chunking on token limit errors.

5. **Verification** (`pkg/verification/`) runs quality checks (untranslated block detection, HTML artifact detection, completeness) and supports multi-LLM consensus polishing.
6. **Output generation** writes translated content back to the target format (EPUB writer, Markdown converter).

LLM Provider Pattern

All LLM providers implement the `LLMClient` interface in `pkg/translator/llm/llm.go`:

```
type LLMClient interface {
    Translate(ctx context.Context, text string, prompt string)
        (string, error)
    GetProviderName() string
}
```

Supported providers:

- API-based: OpenAI, Anthropic, Zhipu, DeepSeek, Qwen, Gemini
- Self-hosted: Ollama (local HTTP), LlamaCpp (executes local binary)
- Distributed: SSH workers for remote processing
- Test: Mock provider

Provider selection uses a factory pattern (`NewLLMTranslatorWithConfig()`) that validates provider/model and creates the appropriate client. Valid models are registered in `ValidModels` map.

Import cycle avoidance: `pkg/translator/llm/` uses a type alias (`type TranslationConfig = translator.TranslationConfig`) to avoid direct import cycles with `pkg/translator/`.

Configuration System

- `internal/config/config.go` defines the canonical config struct with nested sections: `Server`, `Security`, `Translation`, `Preparation`, `Distributed`, `Logging`.
- Config is loaded from JSON files (e.g., `config.json`, files in `internal/working/`).
- Environment variables override file values, especially for secrets and API keys (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `ZHIPU_API_KEY`, `DEEPSEEK_API_KEY`, `JWT_SECRET`).
- `Config.Validate()` checks ports, TLS requirements, JWT secret length (minimum 16 characters), and distributed worker configs.

Distributed Processing

- `pkg/distributed/coordinator.go` manages remote LLM instances across SSH workers with round-robin selection and fallback strategies.
- `pkg/distributed/manager.go` is the top-level orchestrator composing `SSHPool`, `PairingManager`, `VersionManager`, `FallbackManager`, and `DistributedCoordinator`.

- `pkg/sshworker/worker.go` handles SSH connection management, file transfer, remote command execution, and codebase version synchronization.
- `pkg/distributed/version_manager.go` compares local vs. remote codebase versions, supports update/rollback flows, and emits drift alerts.
- `pkg/distributed/fallback.go` tracks per-component failure rates with exponential backoff and degraded mode fallback (remote → local → reduced quality).
- Workers must be reachable from the coordinator. Version synchronization is required.

Markdown Workflow

- `pkg/markdown/epub_to_markdown.go` converts EPUB to Markdown preserving metadata, cover images, and chapter structure.
- `pkg/markdown/markdown_to_epub.go` converts translated Markdown back to EPUB.
- `pkg/markdown/translator.go` translates Markdown line-by-line while preserving syntax (headers, lists, code blocks, links, inline formatting).
- `pkg/markdown/simple_workflow.go` orchestrates the full Ebook → Markdown → Translate → Ebook pipeline.

gRPC and Protobuf

- Protobuf definition lives at `pkg/grpc/translator.proto`.
- Generated code is consumed from `pkg/grpc/proto`.
- `pkg/grpc/server.go` implements `TranslationService` with session management, streaming progress, provider registry, and cleanup routines.

Security Considerations

- **TLS required for production**: The main `server` entry point uses HTTPS/HTTP3. TLS certificates are expected in the `certs/` directory (`server.crt`, `server.key`). Self-signed cert generation is available via `--generate-certs` flag.
- **JWT authentication**: Configurable via `security.enable_auth` in config. Secret must be at least 16 characters. Timing attack mitigation adds an artificial delay for invalid tokens.
- **Rate limiting**: Configurable requests-per-second and burst limits via `security.rate_limit_rps` and `security.rate_limit_burst`. Per-key limiters with background cleanup every 10 minutes.
- **API key management**: API keys must be provided via environment variables (e.g., `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`). Never commit them.
- **CORS**: Configurable allowed origins via `security.cors_origins`.
- **Input validation**: All API endpoints validate request bodies using Gin binding tags (`binding:"required"`, `binding:"email"`, etc.).
- **SSH security**: Key-based authentication is preferred for SSH workers. Password auth is supported but discouraged for production.
- **Transport**: HTTP/3 uses TLS 1.3 with `h3` ALPN; HTTP/2 fallback uses TLS 1.2+.

Sensitive Files (Already in .gitignore)

- `config_with_keys.json`, `api_keys.json`
- `.env`, `.env.*`, `secrets.*`, `keys.*`
- `qwen_credentials.json`, `oauth_creds.json`
- `.translator/` directory

Deployment Processes

Docker Deployment

A multi-stage `Dockerfile` is provided:

- **Builder stage**: `golang:1.24-alpine` with `git` and `make`
- **Production stage**: `alpine:latest` with `ca-certificates` and `tzdata`
- Runs as non-root user (`appuser`, UID 1000)
- Exposes port 8443 (TCP and UDP for HTTP/3)
- Health check endpoint: `https://localhost:8443/health`
- Default command: `/app/translator-server -config /app/config/config.json`
- Volumes exposed for `/app/certs` and `/app/config`

Note: The `Dockerfile` references `make build-server`, which does not exist in the root `Makefile`. The Makefile defines `build`, `build-grpc`, `build-api`, and `build-cli` instead.

Docker Compose

`docker-compose.yml` provides a full production stack:

- PostgreSQL 16 (with health checks and optional init script)
- Redis 7 (with password, 512MB maxmemory, `allkeys-lru` policy)
- Translator API server (depends on healthy postgres & redis)
- Optional admin tools (Adminer, Redis Commander) via `--profile admin`

`docker-compose.distributed.yml` provides a distributed worker stack with main server, worker instances, and Ollama services.

Manual Deployment

```
# Build binaries
make build

# Start gRPC server
./build/grpc-server

# Start API server
./build/api-server
```

```
# Start monitoring server
./build/monitor-server
```

Deployment Scripts

`internal/scripts/` contains extensive operational scripts:

- `deploy-system.sh` - Automated deployment using `./build/deployment-cli`
- `deploy_ssh_worker.sh` / `deploy_worker.sh` - SSH worker deployment
- `check-health.sh` / `check_worker.sh` - Health checks
- `monitor-production.sh` - Full production monitoring with disk/CPU/memory/API/DB/Redis checks and alerting
- `start.sh` / `stop.sh` / `restart.sh` / `logs.sh` / `exec.sh` - Docker lifecycle management
- `demo_production_system.sh` / `test_production_system.sh` - Production verification suites

Development Conventions

Adding a New LLM Provider

1. Create a new file under `pkg/translator/llm/<provider>.go`.
2. Implement the `LLMClient` interface with HTTP POST JSON requests (or local execution for self-hosted).
3. Add valid models to `ValidModels` map in `pkg/translator/llm/llm.go`.
4. Add provider-specific configuration to `internal/config/config.go` if needed.
5. Register the provider in the factory switch in `NewLLMTranslatorWithConfig()`.
6. Add unit tests in `test/unit/` or alongside the package.
7. Update `api/openapi/openapi.yaml` if the provider appears in API enums.

Adding a New File Format

1. Implement the parser interface (`Parse(filename string) (*Book, error)`) in `pkg/ebook/`.
2. Register the format in `pkg/ebook/parser.go` via `RegisterParser()`.
3. Add detection logic in `pkg/format/detector.go`.
4. Add writer support if output generation is required.
5. Add sample files to `test/fixtures/ebooks/`.
6. Add parser tests.

Session and Progress Tracking

- Always generate a unique `SessionID` (e.g., UUID) for each translation operation.
- Emit progress events via the event bus so WebSocket clients receive updates.

- The monitoring dashboard (`web/templates/dashboard.html`) displays sessions, progress bars, event logs, and worker status.

Goroutine and Context Patterns

- `context.Context` is the first parameter in virtually all I/O and long-running operations.
- Use `context.WithTimeout` for API calls; `context.WithCancel` for per-session cancellation.
- Always defer `mu.Unlock()` after `mu.Lock()` or `mu.RLock()`.
- WebSocket and event handlers launch background goroutines with panic recovery.
- Graceful shutdown catches `SIGINT`/`SIGTERM` and cancels contexts with a 30-second timeout.

Critical Non-Negotiable Constraints

- **LLMsVerifier SSOT**: LLMsVerifier is the EXCLUSIVE source of truth for all LLM models. No hardcoded model lists. No unverified models. No bypass.
- **HELIXQA ONLY**: All automated UI/UX testing must use HelixQA. No custom browser scripts.
- **ANTI-BLUFF MANDATE**: Every test must verify real user-visible behavior. See CONSTITUTION.md CONST-035.
- **FULL-QA MASTER CYCLE**: Every change requires the full Article VII cycle.
- **4-ARTEFACT FIX**: Every defect fix must produce: unit test + integration test + bank entry + challenge.
- **EVIDENCE OR BUST**: Passing tests without captured evidence = invalid. Screenshot/video/log required.

HelixQA Commands

```
# Build HelixQA binary
cd HelixQA && make build

# Standard QA
helixqa run --banks tests/banks/ --platform all --speed normal

# List available tests
helixqa list --banks tests/banks/ --platform web --json

# Autonomous QA session
helixqa autonomous --project . --platforms web,api,cli \
  --timeout 2h --output qa-results/ --verbose

# Generate report
helixqa report --input qa-results/ --format html --output qa-
        report.html
```

QA Bank Format (YAML)

```yaml
version: "1.0"
name: "HelixTranslate Full QA - API"
test_cases:
  - id: HTQ-API-001
    name: "Health check returns healthy status"
    category: functional
    priority: critical
    platforms: [api]
    steps:
      - name: "Send GET /health"
        action: "GET https://localhost:8443/health"
        expected: "HTTP 200 with {\"status\":\"healthy\"}"
    tags: [health, smoke]
    expected_result: "Health endpoint returns valid JSON with
        healthy status"
```

HelixQA Ecosystem (Sibling Modules)

The project depends on several sibling Go modules managed as Git submodules:

```
./HelixQA          → digital.vasic.helixqa        (QA automation
engine)
./DocProcessor     → digital.vasic.docprocessor   (Document
processing)
./LLMOrchestrator  → digital.vasic.llmorchestrator (Agent
orchestration)
./LLMProvider      → digital.vasic.llmprovider    (Provider facade
+ circuit breaker)
./VisionEngine     → digital.vasic.visionengine   (Vision + OpenCV
analysis)
./LLMsVerifier     → digital.vasic.llmsverifier   (Model
verification SSOT)
./Challenges       → digital.vasic.challenges     (Challenge
framework)
./Containers       → digital.vasic.containers     (Container
orchestration)
./Models           → digital.vasic.models         (Shared data
types)
./Security         → digital.vasic.security       (Security
utilities / SSRF)
```

All submodules are wired via `replace` directives in `go.mod`. If a submodule fails to clone (e.g. `LLMsVerifier` repository unavailable), create a local placeholder with a matching `go.mod` module name and populate the packages that HelixTranslate imports.

LLMsVerifier Integration Architecture

```
[HelixTranslate Application]
    |
    | 1. Config Load
    v
[internal/verifier/]
    |
    | 2. Verify All Providers
    v
[digital.vasic.llmsverifier/pkg/api]
    |---> types.go (canonical Model, PricingInfo — CONST-034 SSOT)
    |---> pkg/helixqa (VisionModelRegistry for HelixQA ranking)
    |
    | 3. Return Verified + Scored Models
    v
[internal/services/llmsverifier_score_adapter.go]
    |
    | 4. Score Normalization (0–100 -> 0–10)
    v
[pkg/translator/llm/llm.go]
    |
    | 5. Translation via Verified Providers (26 providers)
    v
[LLM Provider APIs]
```

Canonical type location: `llms_verifier/pkg/api/types.go` defines `Model` and `PricingInfo`. HelixTranslate aliases them in `internal/verifier/client.go` (`type Model = api.Model`).

Essential Files Quick Reference

File	Purpose
`go.mod` / `go.sum`	Module definition and dependency checksums
`Makefile`	Build, test, and development commands
`config.json`	Default application configuration
`Dockerfile`	Multi-stage container build
`docker-compose.yml`	Full stack with PostgreSQL and Redis
`.golangci.yml`	Strict linting configuration
`internal/config/config.go`	Canonical configuration structs and loading
`pkg/events/events.go`	Central pub/sub event system
`pkg/websocket/hub.go`	WebSocket hub for real-time monitoring
`pkg/translator/llm/llm.go`	LLM provider interface and factory (26 providers)
`llms_verifier/pkg/api/types.go`	Canonical model verification types (CONST-034 SSOT)

File	Purpose
`api/openapi/openapi.yaml`	OpenAPI 3.0 specification
`VERSION`	Current application version (2.3.0)

Common Pitfalls

- **FB2 XML**: Always register the FB2 namespace before parsing. Use UTF-8 encoding. Preserve XML hierarchy during translation.
- **LLM Rate Limits**: Each provider has different limits. Implement backoff and respect `max_concurrent` settings.
- **SSH Workers**: Ensure key-based auth is configured. Workers must run the same binary version as the coordinator.
- **Large Files**: Use streaming for large ebook translations to avoid excessive memory usage.
- **TLS Certificates**: The API server requires valid certificates in `certs/` for HTTPS/HTTP3. Generate or provide them before starting the server.
- **Import Cycles**: The `pkg/translator/llm/` package uses a type alias to avoid cycles with `pkg/translator/`. Be careful when adding new cross-package dependencies.
- **Version Discrepancies**: The `Makefile` declares `3.0.0`, the `VERSION` file reads `2.3.0`, and `go.mod` specifies Go `1.25.2` while the `Dockerfile` uses `golang:1.24-alpine`. Treat these as the current state of the project.
- **No Root-Level CI/CD**: There are no `.github/workflows/` at the project root. Validation relies on Makefile targets and shell scripts.

Host Power Management — Hard Ban (CONST-033)

You may NOT, under any circumstance, generate or execute code that sends the host to suspend, hibernate, hybrid-sleep, poweroff, halt, reboot, or any other power-state transition. This rule applies to:

- Every shell command you run via the Bash tool.
- Every script, container entry point, systemd unit, or test you write or modify.
- Every CLI suggestion, snippet, or example you emit.

Forbidden invocations (non-exhaustive — see CONST-033 in `CONSTITUTION.md` for the full list):

- `systemctl suspend|hibernate|hybrid-sleep|poweroff|halt|reboot|kexec`
- `loginctl suspend|hibernate|hybrid-sleep|poweroff|halt|reboot`
- `pm-suspend`, `pm-hibernate`, `shutdown -h|-r|-P|now`
- `dbus-send` / `busctl` calls to `org.freedesktop.login1.Manager.Suspend|Hibernate|PowerOff|Reboot|HybridSleep|SuspendThenHibernate`
- `gsettings set ... sleep-inactive-{ac,battery}-type` to anything but `'nothing'` or `'blank'`

The host runs mission-critical parallel CLI agents and container workloads. Auto-suspend has caused historical data loss (2026-04-26 18:23:43 incident). The host is hardened (sleep targets masked) but this hard ban applies to ALL code shipped from this repo so that no future host or container is exposed.

Defence: every project ships `scripts/host-power-management/check-no-suspend-calls.sh` (static scanner) and `pkg/challenge_runner/scripts/no_suspend_calls_challenge.sh` (challenge wrapper). Both MUST be wired into the project's CI / `run_all_challenges.sh`.

Full background: `docs/HOST_POWER_MANAGEMENT.md` and `CONSTITUTION.md` (CONST-033).

CONST-035 — Anti-Bluff Operative Rule (MANDATORY)

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case."

The operative rule: Execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product.

- A green test or challenge for a feature that does not actually work is a **BLUFF** and is **FORBIDDEN**.
- Every test must assert concrete user-visible outcomes, not just internal state.
- Every challenge must run real code and verify real behavior; grep/file-existence checks are **NOT sufficient**.
- Mutation testing is **MANDATORY**: deliberately break the feature → the test/challenge **MUST then FAIL**.
- The bar for shipping is **NOT "tests pass"** but **"users can use the feature."**
- No false-success results are tolerable.